

lab2问答清单

1. 什么是实模式，什么是保护模式？
2. 什么是选择子？
3. 什么是描述符？
4. 什么是GDT，什么是LDT？
5. 请分别说明GDTR和LDTR的结构。
6. 请说明GDT直接查找物理地址的具体步骤。
7. 请说明通过LDT查找物理地址的具体步骤。
8. 根目录区大小一定么？扇区号是多少？为什么？
9. 数据区第一个簇号是多少？为什么？
10. FAT表的作用
11. 解释静态链接的过程。
12. 解释动态链接的过程。
13. 静态链接相关PPT中为什么使用ld链接而不是gcc
14. linux下可执行文件的虚拟地址空间默认从哪里开始分配。

实验相关内容

1. BPB指定字段的含义
2. 如何进入子目录并输出（说明方法调用）
3. 如何获得指定文件的内容，即如何获得数据区的内容（比如使用指针等）
4. 如何进行C代码和汇编之间的参数传递和返回值传递
5. 汇编代码中对I/O的处理方式，说明指定寄存器所存值的含义

1. 什么是实模式，什么是保护模式？

实模式：用基地址加偏移量就可以直接拿到物理地址的模式。缺点是不安全；

保护模式：不能直接拿到物理地址的模式，需要进行地址转换，从80386开始是现代操作系统的主要模式

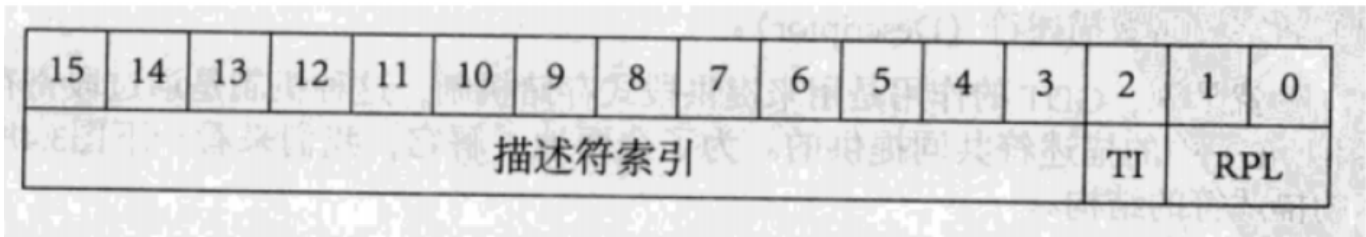
2. 什么是选择子？

选择子共16位，放在段选择寄存器里

低2位表示请求特权级

第3位表示选择GDT方式还是LDT方式

高13位表示在描述符表中的偏移（故描述符表的项数最多是2的 13次方）



3.什么是描述符？

在保护模式下引入的来描述各种数据段的二进制码，大小为8个字节，第5个字节说明描述符的类型



4. 什么是GDT，什么是LDT？

GDT：全局描述符表，是全局唯一的，用来存放一些公用的描述符和包含各进程局部描述符表首地址的描述符

LDT：局部描述符表，每个进程都可以有一个，用于存放本进程内使用的描述符

5. 请分别说明GDTR和LDTR的结构。

GDTR：是48位寄存器，高32位放置GDT首地址，低16位放置GDT限长（决定了可寻址大小）

LDTR：是16位寄存器，放置一个特殊的选择子，用于查找当前进程的LDT首地址

6. 请说明GDT直接查找物理地址的具体步骤。

- (1) 给出段选择子+偏移量
- (2) 若选择了GDT方式，则从GDTR获取GDT首地址，用段选择子中的13位做偏移，拿到GDT中的描述符
- (3) 如果合法且有权限，用描述符中的段首地址+偏移量找到物理地址

7. 请说明通过LDT查找物理地址的具体步骤。

- (1) 给出段选择子+偏移量
- (2) 若选择了LDT方式，则从GDTR获取GDT首地址，用LDTR中的偏移量做偏移，拿到GDT中的描述符1
- (3) 从描述符1中获取LDT首地址，用段选择子中的13位做偏移，拿到LDT中的描述符2
- (4) 如果合法且有权限，用描述符2中的段首地址+偏移量找到物理地址

8. 根目录区大小一定么？扇区号是多少？为什么？

长度不固定，需要计算

数据区第一个簇即第一个扇区在哪里呢？参照图4.1我们发现，必须计算根目录区所占的扇区数才能知道。我们既然已经知道了根目录区条目最多有BPB_RootEntCnt个，扇区数也就容易计算了，假设根目录区共占用RootDirSectors个扇区，则有：

$$RootDirSectors = \frac{(BPB_RootEntCnt \times 32) + (BPB_BytsPerSec - 1)}{BPB_BytsPerSec}$$

9. 数据区第一个簇号是多少？为什么？

2, 因为FAT表的0和1项始终不使用, 从第二个FAT项开始表示数据区的第一个簇, 为了方便表示就用2当作数据区第一个簇号是多少?

10. FAT表的作用

用来保存包含文件簇信息的表项, 与数据区中的簇构成对应关系, 实现文件的链式存储。

- 大于0xFF8, 说明是最后一个簇
- 等于0xFF7, 说明是坏簇
- 其他, 指明了大文件的下一个数据区所在的簇

11. 解释静态链接的过程。

静态链接是指在编译阶段直接把静态库加入到可执行文件中, 这样可执行文件会比较大。

(1) 空间与地址分配 – 扫描所有目标文件, 获得各个段的长度、属性和起始地址, 合并各个目标文件的符号表, 得到一个全局符号表, 相似段合并得到所有目标文件的段长度、位置等信息, 此后的虚拟地址(函数、变量)就可以以此确定了

(2) 符号解析和重定位 – 根据全局符号表对符号的引用进行解析和重定位; 符号解析就是找到符号的正确地址; 重定位就是修正指令对引用其他目标文件的函数或变量的地址(未确定之前用的都是假地址), 根据(1)中的全局符号表就能够重定位所有的符号。

12. 解释动态链接的过程。

动态链接则是指链接阶段仅仅只加入一些描述信息, 而程序执行时再从系统中把相应动态库加载到内存中去。

动态链接器自举、装载共享对象、重定位和初始化、转交控制权

13. 静态链接相关PPT中为什么使用ld链接而不是gcc

ld对C或任何其他语言一无所知. ld不知道你的代码需要链接哪些库.如果你试图直接将已编译的C代码与ld链接起来,那么它会拯救你,因为ld本身并不知道它在哪里可以找到libstdc,即gcc的C运行时库.你用字符串吗? 引导? 其中大部分是模板代码,它被编译为对象模块的一部分.但是在libstdc中仍然有一些需要链接的预编译位.

比如使用ld就会

```
ld -m elf_i386 -o main main.o my_print.o
ld: 警告: 无法找到项目符号 _start; 缺省为 00000000080480c0
main.o: 在函数 'main' 中:
main.cpp: (.text+0x19): 对 'fopen' 未定义的引用
main.cpp: (.text+0x42): 对 'malloc' 未定义的引用
main.cpp: (.text+0x71): 对 'std::cin' 未定义的引用
main.cpp: (.text+0x76): 对 'std::istream::getline(char*, int)' 未定义的引用
main.cpp: (.text+0x8a): 对 'strcmp' 未定义的引用
main.cpp: (.text+0x10b): 对 'strlen' 未定义的引用
main.o: 在函数 'doCAT(_IO_FILE*, char*)' 中:
main.cpp: (.text+0x140): 对 'malloc' 未定义的引用
main.cpp: (.text+0x155): 对 'strcpy' 未定义的引用
main.cpp: (.text+0x169): 对 'malloc' 未定义的引用
main.cpp: (.text+0x17e): 对 'std::allocator<char>::allocator()' 未定义的引用
main.cpp: (.text+0x197): 对 'std::basic_string<char, std::char_traits<char>, std::
allocator<char> >::basic_string(char const*, std::allocator<char> const&)' 未定义
的引用
main.cpp: (.text+0x1a2): 对 'std::allocator<char>::allocator()' 未定义的引用
main.cpp: (.text+0x1bb): 对 'std::basic_string<char, std::char_traits<char>, std::
allocator<char> >::basic_string(char const*, std::allocator<char> const&)' 未定义
的引用
```

14. linux下可执行文件的虚拟地址空间默认从哪里开始分配。

0x08048000

实验相关内容

1. BPB指定字段的含义

名称	偏移	长度	内容	Orange'S的值
BS_jmpBoot	0	3	一个短跳转指令	jmp LABEL_START nop
BS_OEMName	3	8	厂商名	'ForrestY'
BPB_BytsPerSec	11	2	每扇区字节数	0x200
BPB_SecPerClus	13	1	每簇扇区数	0x1
BPB_RsvdSecCnt	14	2	Boot 记录占用多少扇区	0x1
BPB_NumFATs	16	1	共有多少 FAT 表	0x2
BPB_RootEntCnt	17	2	根目录文件数最大值	0xE0
BPB_TotSec16	19	2	扇区总数	0xB40
BPB_Media	21	1	介质描述符	0xF0
BPB_FATSz16	22	2	每 FAT 扇区数	0x9
BPB_SecPerTrk	24	2	每磁道扇区数	0x12
BPB_NumHeads	26	2	磁头数 (面数)	0x2
BPB_HiddSec	28	4	隐藏扇区数	0
BPB_TotSec32	32	4	如果BPB_TotSec16是0, 由这个值记录扇区数	0
BS_DrvNum	36	1	中断 13 的驱动器号	0
BS_Reserved1	37	1	未使用	0
BS_BootSig	38	1	扩展引导标记 (29h)	0x29
BS_VolID	39	4	卷序列号	0
BS_VolLab	43	11	卷标	'OrangeS0.02'
BS_FileSysType	54	8	文件系统类型	'FAT12'
引导代码及其他	62	448	引导代码、数据及其他填充字符等	引导代码 (剩余空间被0填充)
结束标志	510	2	0xAA55	0xAA55

2. 如何进入子目录并输出 (说明方法调用)

调用方法

```

1  FileAndDir *locateFD(FileAndDir *currentDir, char *filePath)
2  {
3      FileAndDir *res = currentDir;
4      char *target = (char *)malloc(sizeof(char) * 64);
5      vector<string> v = split(filePath, "/");
6      for (int i = 0; i < v.size(); ++i)
7      {
8          strcpy(target, v.at(i).c_str());
9          res = res->locate(target);
10         if (res == NULL)
11         {
12             break; // 如果找不到就设为NULL，直接返回，外层再处理
13         }
14     }
15     return res;
16 }

```

递归寻找

```

1  FileAndDir *locate(char *target)
2  {
3      for (vector<FileAndDir *>::iterator it = subFDs.begin(); it != sub
4      FDs.end(); ++it)
5      {
6          FileAndDir *fd = *it;
7          if (strcmp(target, fd->name) == 0)
8          {
9              return fd;
10         }
11     }
12     return NULL;

```

然后根据输出要求，调用LS_L或者LS或CAT

3. 如何获得指定文件的内容，即如何获得数据区的内容（比如使用指针等）

递归

```

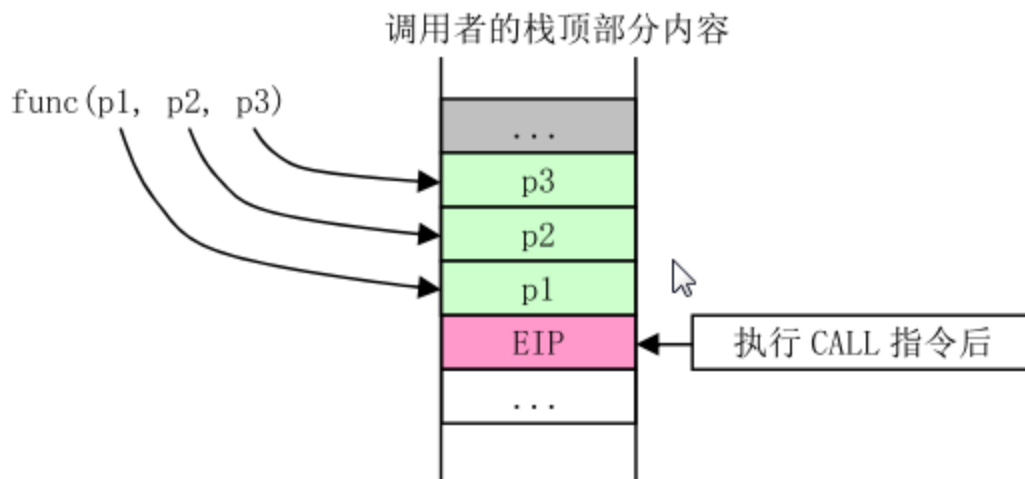
1  int offset = DataBaseSectors * bpb_ptr->BPB_BytsPerSec;
2  int currentClus = this->startClus;
3  int bytsPerClus = bpb_ptr->BPB_BytsPerSec * bpb_ptr->BPB_SecPerClus;
4  while (true)
5  {
6      int nextClus = readFAT(fat12, currentClus);
7      if (nextClus == 0xFF7)
8      {
9          temp = "Error: bad clus.";
10         my_printWhite(temp, strlen(temp));
11         break; // 坏簇
12     }
13     int startByte = offset + (currentClus - 2) * bytsPerClus;
14     temp = (char *)malloc(bytsPerClus);
15     loadFile(fat12, startByte, bytsPerClus, temp);
16     strcat(temp, "\0");
17     my_printWhite(temp, strlen(temp));
18     delete temp; // 避免内存泄漏
19     currentClus = nextClus;
20     if (nextClus >= 0xFF8)
21     {
22         break; // end
23     }
24 }

```

4. 如何进行C代码和汇编之间的参数传递和返回值传递

C 中的函数参数被入栈，在汇编中根据 esp 计算参数在栈中的位置

返回值被放在 eax 中



调用函数时压入堆栈的参数

```

1 void my_printRed(char *str, int length); //打印
2 void my_printWhite(char *str, int length); //打印

```

依次压入长度，指针，返回地址

在汇编中

```

1 mov eax,4 ;系统调用号
2 mov ebx,1 ;文件描述符stdout
3 mov ecx,[esp+4] ;起始地址
4 mov edx,[esp+8] ;打印长度
5 int 80h

```

5. 汇编代码中对I/O的处理方式，说明指定寄存器所存值的含义

输出同上

通过 esp 计算传入的参数位置，eax 存系统调用号，ebx 存文件描述符，ecx 存 起始地址，edx 存长度

输入

```
1      ;接收输入
2      mov eax,3
3      mov ebx,0
4      mov ecx,input
5      mov edx,128
6      int 80h
```